Yale University

Optimization for Deep Learning

S&DS 265 · Introductory Machine Learning · Lecture 12

Zhuoran Yang

Fall 2026

AI usage: figures were generated, and these slides polished, with the help of AI tools.

Outline

1. Data Order, Minibatches, and Epochs
2. From Momentum to AdamW
3. Matrix-Aware Optimizers
4. The PyTorch Training Loop

Learning Objectives

In this lecture you will learn:

1. How shuffling, minibatches, optimizer steps, and epochs define a training trajectory;
2. Why AdaGrad, RMSProp, Adam, and AdamW store different gradient histories;
3. Why matrix-aware updates such as Shampoo, SOAP, and Muon can help, and what a run keeps in memory;
4. How PyTorch's Dataset, DataLoader, and Optimizer classes implement these pieces;
5. How to choose and report a learning-rate schedule in the correct unit.

Outline

1. Data Order, Minibatches, and Epochs
2. From Momentum to AdamW
3. Matrix-Aware Optimizers
4. The PyTorch Training Loop

Motivating Example: How Should Parameters Move?

Lecture 11 supplied the model, loss, initialization, and gradients. Now we must decide which examples define the next gradient, how that gradient becomes an update, and how the learning rate changes.

The data stream and update rule together define the training algorithm.

Example: training the MNIST MLP from Lecture 11.

The Training Dataset

Store n labeled observations as a map

$$i \longmapsto (x_i, y_i), \qquad i = 0, \ldots, n-1.$$

A PyTorch `Dataset` answers "what is observation i?" A `DataLoader` answers "which indices form the next batch, and how are their tensors assembled?"

These are separate from the model, loss, and optimizer.

Reading One Image Batch

For MNIST with batch size $B = 128$, one loader output has

$$X_{\mathcal{B}} : [128, 1, 28, 28], \qquad y_{\mathcal{B}} : [128].$$

The MLP flattens the images to $[128, 784]$ before the first linear layer.

One batch determines one gradient estimate

$$g_{\mathcal{B}}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_\theta \ell_i(\theta).$$

Problem Setup

Hold the data, MLP class, and empirical objective fixed:

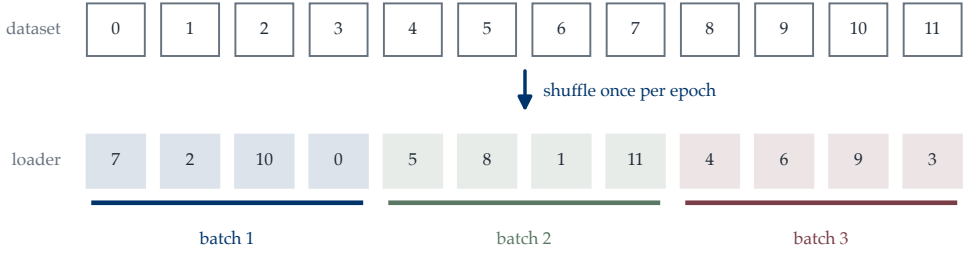
$$\widehat{R}_n(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell_i(\theta).$$

Today's fitted estimator is the output of a **training procedure** applied to the same model class:

$$\hat{\theta}_T = \mathcal{A}_T(\theta_0; \mathcal{B}_{1:T},\ \eta_{1:T},\ \text{state}).$$

The loader supplies $\mathcal{B}_t$, autodiff estimates g_t, and the optimizer updates its state and θ_t. The objective is fixed; changing batch order, update geometry, or schedule can produce a different fitted parameter.

Shuffling Forms Batches Without Replacement



Every observation appears once; batch neighbors change after reshuffling.

Source: Declared permutation.

This Differs Slightly from the Lecture 5 Idealization

Lecture 5 analyzed the convenient with-replacement model

$$I_t \sim \text{Uniform}\{1, \dots, n\}$$

independently at every step. A common implementation instead draws one without-replacement permutation per epoch and walks through it in batches.

- Within an epoch, examples are sampled without replacement.
- Across epochs, a new permutation changes batch composition.
- Both produce useful stochastic gradients, but their dependence structures differ.

An Epoch Is One Pass through the Loader

If `drop_last=False`, the number of batches is

$$\boxed{\text{steps per epoch} = \left\lceil \frac{n}{B} \right\rceil}.$$

For MNIST, $n = 60{,}000$ and $B = 128$ give 469 optimizer steps per epoch, so E epochs process about En examples but make only $E\lceil n/B \rceil$ updates.



Always say whether a schedule is indexed by epochs, batches, or optimizer steps.

A Dataset Maps an Index to One Example

```
from torch.utils.data import Dataset

class MNISTTensors(Dataset):
    def __init__(self, images, labels):      # images [n,28,28] uint8, labels [n]
        self.images = images.float() / 255
        self.labels = labels

    def __len__(self):                        # n
        return len(self.labels)

    def __getitem__(self, i):                 # i -> (x_i, y_i)
        return self.images[i].unsqueeze(0), self.labels[i]   # [1,28,28], []
```

The two methods are the whole contract: `__len__` returns n and `__getitem__` returns (x_i, y_i). Work done one example at a time belongs here, such as pixel scaling or a random crop. The dataset knows nothing about batches, order, or epochs.

What a DataLoader Does in One Epoch

```
def one_epoch(dataset, B, generator, drop_last=False):
    perm = torch.randperm(len(dataset), generator=generator)  # 1. sampler
    for start in range(0, len(dataset), B):
        idx = perm[start:start + B].tolist()                  # 2. batch sampler
        if drop_last and len(idx) < B:
            return
        xs, ys = zip(*[dataset[i] for i in idx])              # 3. __getitem__
        yield torch.stack(xs), torch.stack(ys)                # 4. collate
```

`DataLoader(dataset, batch_size=B, shuffle=True)` runs these four stages. The sampler draws one fresh permutation each time iteration begins, which is the without-replacement scheme of the previous slides; the collate function stacks B examples of shape $[1, 28, 28]$ into one tensor $[B, 1, 28, 28]$.

The Real Loader Adds Workers and a Generator

```
train_dataset = MNISTTensors(train_images, train_labels)
loader = DataLoader(
    train_dataset,
    batch_size=128,
    shuffle=True,                                  # new permutation per epoch
    generator=torch.Generator().manual_seed(0),    # which permutations
    drop_last=False,                               # keep the short batch
    num_workers=4,                                 # stage 3 in 4 processes
    pin_memory=True,                               # faster copy to the GPU
)
```

The first four arguments decide which examples form each batch, so they change the training trajectory. The last two decide only how fast the batches arrive: workers call `__getitem__` in parallel while the GPU computes, and pinned memory speeds the host-to-device copy.

The Last Batch, Counted for MNIST

With $n = 60{,}000$ and $B = 128$,

$$60{,}000 = 468 \times 128 + 96.$$

	`drop_last=False`	`drop_last=True`
steps per epoch	$469 = \lceil n/B \rceil$	$468 = \lfloor n/B \rfloor$
last batch	96 examples	none
examples used per epoch	all 60,000	59,904; a different 96 left out each epoch

Keeping the short batch costs little: its gradient averages 96 examples, so its variance is about $128/96 \approx 1.3$ times that of a full batch, for one step in 469. Drop it when every batch must have the same shape, as for compiled kernels, or when a layer such as batch normalization estimates statistics from the batch and a small final batch would give a poor estimate.

Reproducibility: Seed Every Source of Randomness

source of randomness	how to fix it
initialization and dropout masks	`torch.manual_seed(0)`
shuffling order	`generator=torch.Generator().manual_seed(0)`
augmentation inside workers	`worker_init_fn` seeds NumPy and `random`
GPU kernels with atomic additions	`torch.use_deterministic_algorithms(True)`

```
def seed_worker(worker_id):              # PyTorch seeds its own RNG per worker
    s = torch.initial_seed() % 2**32
    numpy.random.seed(s); random.seed(s)
```

Test it directly: two runs with the same seeds should print identical losses for the first few steps. With several GPUs, a `DistributedSampler` splits each permutation across devices and needs `sampler.set_epoch(e)` at every epoch; without that call, every epoch repeats the first epoch's order.

Outline

1. Data Order, Minibatches, and Epochs
2. From Momentum to AdamW
3. Matrix-Aware Optimizers
4. The PyTorch Training Loop

The Optimizer Converts a Gradient into Parameter Motion

Backpropagation provides $g_t = \nabla_\theta L_{\mathcal{B}_t}(\theta_t)$. An optimizer maintains state s_t and applies an update rule

$$(\theta_{t+1}, s_{t+1}) = \mathcal{U}(\theta_t, g_t, s_t; \eta_t).$$

Different algorithms use the same gradients but retain different histories:

- Momentum smooths recent directions;
- AdaGrad and RMSProp estimate coordinatewise scale;
- Adam combines both; AdamW adds decoupled shrinkage;
- Shampoo and Muon use matrix structure for selected weights.

SGD and Momentum Are the Starting Point

Lecture 5 introduced

$$\theta_{t+1} = \theta_t - \eta_t g_t$$

and momentum

$$m_t = \beta m_{t-1} + (1-\beta) g_t, \qquad \theta_{t+1} = \theta_t - \eta_t m_t.$$

Momentum smooths directions across batches, but all coordinates still share one scalar learning rate. Adaptive methods change this geometry.

Why One Learning Rate Can Be Slow

Consider the two-dimensional quadratic

$$f(\theta_1, \theta_2) = \tfrac{1}{2}(100\theta_1^2 + \theta_2^2), \qquad \nabla f(\theta) = (100\theta_1, \theta_2).$$

Ordinary gradient descent uses the same η for both coordinates. Stability in the steep θ_1 direction requires $\eta < 2/100$; then progress in the flat θ_2 direction is slow.

Idea: scale coordinates separately so that a large gradient history reduces steps in one coordinate without slowing every coordinate.

AdaGrad Accumulates Coordinatewise Gradient Scale

Keep a running sum of squared gradients, one number per coordinate, and divide the step by its square root:

$$v_t = v_{t-1} + g_t^2, \qquad \theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{v_t} + \varepsilon}.$$

AdaGrad Accumulates Coordinatewise Gradient Scale

Keep a running sum of squared gradients, one number per coordinate, and divide the step by its square root:

$$v_t = v_{t-1} + g_t^2, \qquad \theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{v_t} + \varepsilon}.$$

A coordinate with consistently large gradients accumulates a large v_t and therefore takes smaller steps; a flat coordinate keeps its own scale. One global η now serves both.

AdaGrad Accumulates Coordinatewise Gradient Scale

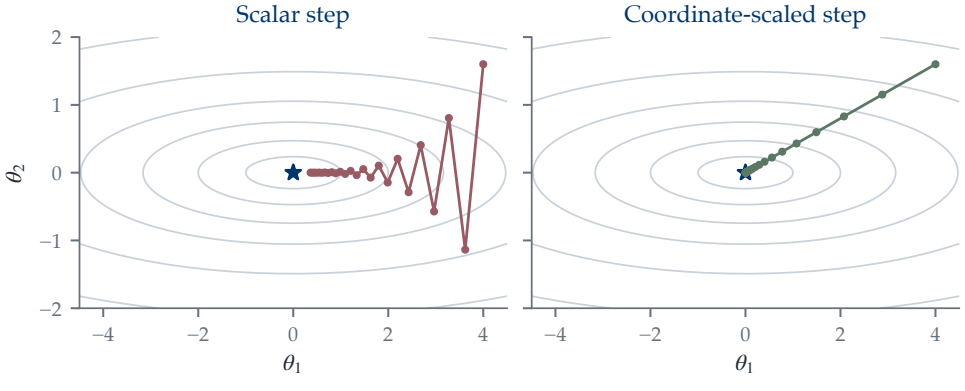
Keep a running sum of squared gradients, one number per coordinate, and divide the step by its square root:

$$v_t = v_{t-1} + g_t^2, \qquad \theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{v_t} + \varepsilon}.$$

A coordinate with consistently large gradients accumulates a large v_t and therefore takes smaller steps; a flat coordinate keeps its own scale. One global η now serves both.

$v_{t,j}$ only grows, so the effective rate $\eta/(\sqrt{v_{t,j}} + \varepsilon)$ can shrink too far. RMSProp discounts old gradients to avoid this accumulation.

One Rate per Coordinate Straightens the Path



The same ill-conditioned quadratic under both rules. A single scalar step must be small enough for the steep coordinate, so it zig-zags and crawls along the flat one; scaling each coordinate by its own accumulated gradient walks almost straight to the minimum.

Course calculation on a declared ill-conditioned quadratic.

RMSProp Forgets Distant Squared Gradients

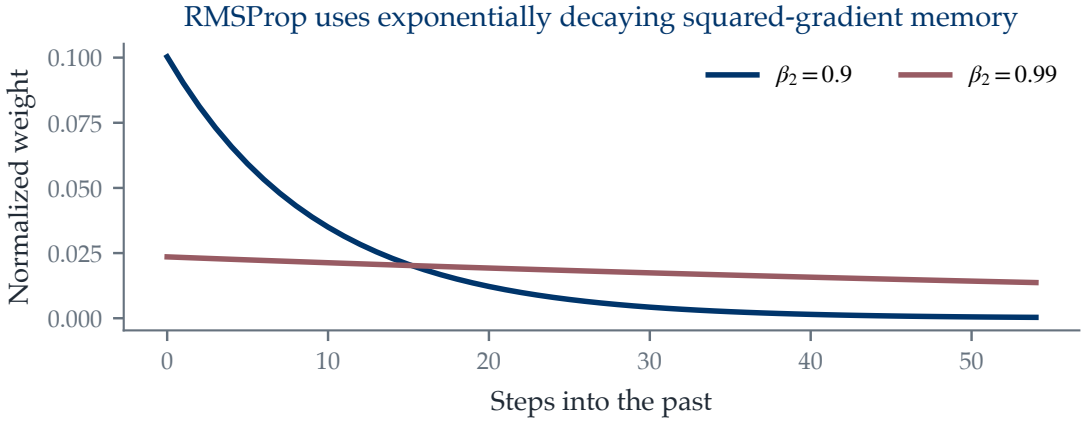
Replace AdaGrad's cumulative sum by an exponential moving average:

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \qquad \theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{v_t} + \varepsilon}.$$

The memory horizon is controlled by β_2; recent gradient scale matters more than distant history. Unlike AdaGrad's sum, it does not retain the full accumulation. Adam adds a moving average of g_t for direction.

Source: D2L, Optimization Algorithms: RMSProp.

Exponential Memory Emphasizes Recent Steps



Course calculation of normalized exponential weights.

Adam Combines Momentum and RMSProp

$$
\begin{aligned}
m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t, \\
v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \\
\widehat{m}_t &= m_t / (1-\beta_1^t), \qquad \hat{v}_t = v_t / (1-\beta_2^t), \\
\theta_{t+1} &= \theta_t - \eta_t \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon}.
\end{aligned}
$$

The first moment smooths direction; the second moment rescales coordinates.

Source: Kingma and Ba, 2015.

Who Is Adam?



A NeurIPS 2025 review flagged the sentence “Both architectures are optimized with Adam” as a serious typo the authors should have removed.

Now we know who Adam is: adaptive moment estimation, the momentum-plus-RMSProp rule on the previous slide, and the default optimizer for training neural networks since Kingma and Ba (2015).

Source: Yiping Lu (@2prime_PKU) on X, shared by Yexi Jiang on LinkedIn, “Who is Adam?!”, July 2025.

Why Adam Corrects Its Moving Averages

Adam starts with $m_0 = v_0 = 0$. After one gradient g_1, its two averages are

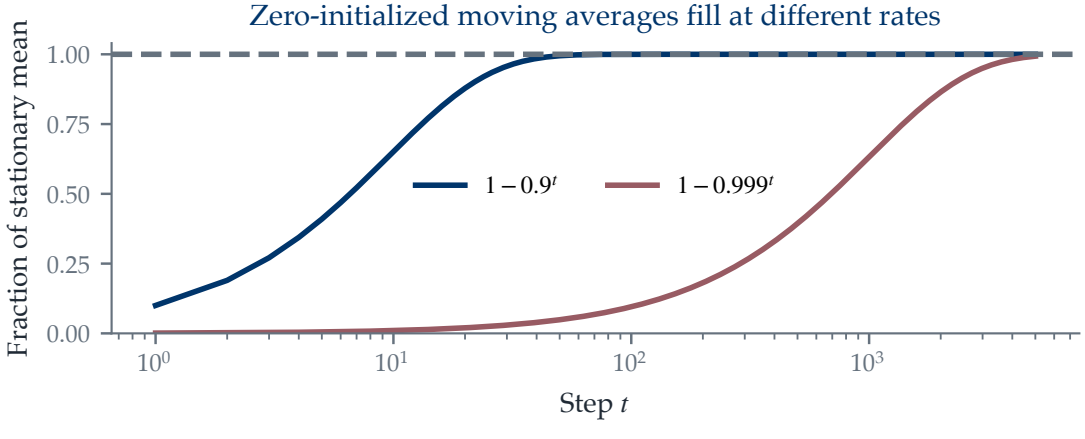
$$m_1 = (1 - \beta_1) g_1, \qquad v_1 = (1 - \beta_2) g_1^2.$$

Dividing by the weight each average has accumulated gives

$$\widehat{m}_1 = \frac{m_1}{1 - \beta_1} = g_1, \qquad \hat{v}_1 = \frac{v_1}{1 - \beta_2} = g_1^2.$$

At step t, the accumulated weights are $1 - \beta_1^t$ and $1 - \beta_2^t$. As $t \to \infty$, both factors approach 1 and the correction fades, while m_t and v_t keep averaging over many recent steps.

Bias Correction Matters Most at the Beginning



Course calculation of moving-average fill-in.

One Adam Step Reveals the Coordinate Scaling

Bias correction has a striking consequence at the first step. With $g_1 = (2, 0.5)$, $\widehat{m}_1 = g_1$ and $\hat{v}_1 = g_1^2$, so, ignoring ε,

$$\Delta\theta_1 = -\eta \frac{(2, 0.5)}{\sqrt{(2, 0.5)^2}} = -\eta(1, 1).$$

Both coordinates move by exactly η, although one gradient is four times the other.

One Adam Step Reveals the Coordinate Scaling

Bias correction has a striking consequence at the first step. With $g_1 = (2, 0.5)$, $\widehat{m}_1 = g_1$ and $\hat{v}_1 = g_1^2$, so, ignoring ε,

$$\Delta\theta_1 = -\eta \frac{(2, 0.5)}{\sqrt{(2, 0.5)^2}} = -\eta(1, 1).$$

Both coordinates move by exactly η, although one gradient is four times the other. Is that a problem? Partly.

- Good: the step is independent of the gradient's units, so one η serves every layer.
- Risky: $\hat{v}_1$ rests on a single noisy gradient, so even a coordinate whose gradient is pure noise moves by the full η.

The usual remedy is a **warmup**: a small η while v_t is still a poor estimate.

Source: Liu et al. (2020), "On the Variance of the Adaptive Learning Rate and Beyond".

Recall: L2 Regularization from Lecture 4

Ridge regression charged for the size of the coefficients,

$$L_\lambda(\beta) = \frac{1}{2n}\|y - X\beta\|_2^2 + \frac{\lambda}{2}\|\beta\|_2^2,$$

and nothing about the penalty needs a linear model. For a network with parameters θ, minimize $\widehat{R}_n(\theta) + \frac{\lambda}{2}\|\theta\|_2^2$: among parameter settings that fit comparably, prefer the one of smaller norm.

Recall: L2 Regularization from Lecture 4

Ridge regression charged for the size of the coefficients,

$$L_\lambda(\beta) = \frac{1}{2n}\|y - X\beta\|_2^2 + \frac{\lambda}{2}\|\beta\|_2^2,$$

and nothing about the penalty needs a linear model. For a network with parameters θ, minimize $\widehat{R}_n(\theta) + \frac{\lambda}{2}\|\theta\|_2^2$: among parameter settings that fit comparably, prefer the one of smaller norm.

One gradient step on the penalized objective, with $g_t = \nabla\widehat{R}_n(\theta_t)$ estimated on a minibatch:

$$\theta_{t+1} = \theta_t - \eta_t(g_t + \lambda\theta_t) = \underbrace{(1 - \eta_t\lambda)\,\theta_t}_{\text{shrink}} - \eta_t g_t.$$

Every step first multiplies the weights by $1 - \eta_t\lambda < 1$, then follows the data gradient. The weights decay, which is why deep learning calls this **weight decay**.

Two Ways to Apply the Penalty

The last equation suggests two implementations of one idea:

1. **L2 in the loss**: hand the optimizer the penalized gradient $g_t + \lambda\theta_t$;
2. **decoupled weight decay**: shrink $\theta_t \mapsto (1 - \eta_t\lambda)\theta_t$ directly, and hand the optimizer only g_t.

For SGD the two give the identical update of the previous slide. An adaptive method, however, rescales whatever gradient it receives, and that includes the term $\lambda\theta_t$.

Giving Adam the gradient of $L_{\text{data}}(\theta) + \frac{\lambda}{2}\|\theta\|_2^2$ is valid L2 regularization, but it does not implement the uniform shrink.

Why L2 in the Loss Is Not Weight Decay for Adam

When we give Adam the penalized gradient $g_t + \lambda\theta_t$, both moment histories include the penalty:

$$g_s^{\text{reg}} = g_s + \lambda\theta_s, \qquad \theta_{t+1} = \theta_t - \eta \frac{\widehat{m}_t^{\text{reg}}}{\sqrt{\hat{v}_t^{\text{reg}}} + \varepsilon}.$$

Why L2 in the Loss Is Not Weight Decay for Adam

When we give Adam the penalized gradient $g_t + \lambda\theta_t$, both moment histories include the penalty:

$$g_s^{\text{reg}} = g_s + \lambda\theta_s, \qquad \theta_{t+1} = \theta_t - \eta \frac{\widehat{m}_t^{\text{reg}}}{\sqrt{\widehat{v}_t^{\text{reg}}} + \varepsilon}.$$

There is no separate $(1 - \eta\lambda)\theta_t$ factor. The penalty's effect depends on each coordinate's gradient history through the first and second moments.

Inside Adam the penalty acts as L2 regularization whose strength differs from coordinate to coordinate; a uniform shrink needs a separate step.

AdamW Decouples the Decay

Take the penalty out of the loss and apply it directly to the parameter, outside the adaptive update:

$$\theta_{t+1} = \underbrace{(1 - \eta_t \lambda)\theta_t}_{\text{uniform shrink}} - \eta_t \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon}$$

AdamW Decouples the Decay

Take the penalty out of the loss and apply it directly to the parameter, outside the adaptive update:

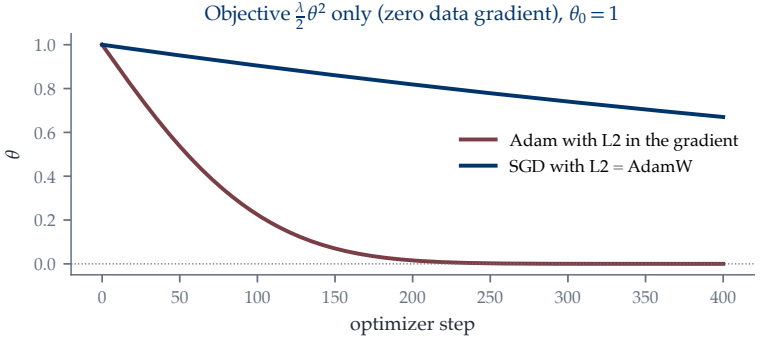
$$\theta_{t+1} = \underbrace{(1 - \eta_t \lambda)\theta_t}_{\text{uniform shrink}} - \eta_t \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon}$$

Within a parameter group, the decay term shrinks every coordinate by $1 - \eta_t \lambda$ per step, independent of its gradient history.

Parameter groups allow different decay rates: for example, apply $\lambda > 0$ to weight matrices and $\lambda = 0$ to biases or normalization scales.

Source: Loshchilov and Hutter (2019).

Coupled L2 and Decoupled Decay Follow Different Paths



One weight, zero data loss: the objective is the penalty $\frac{\lambda}{2}\theta^2$ alone, with $\eta = 0.01$, $\lambda = 0.1$. Blue (AdamW, and SGD alike) shrinks by $1 - \eta\lambda$ per step and keeps 0.67 after 400 steps. Red hands Adam the gradient $\lambda\theta_t$, which Adam divides by its own scale: early steps move about η, ten times AdamW's, and θ is at 0.22 by step 100.

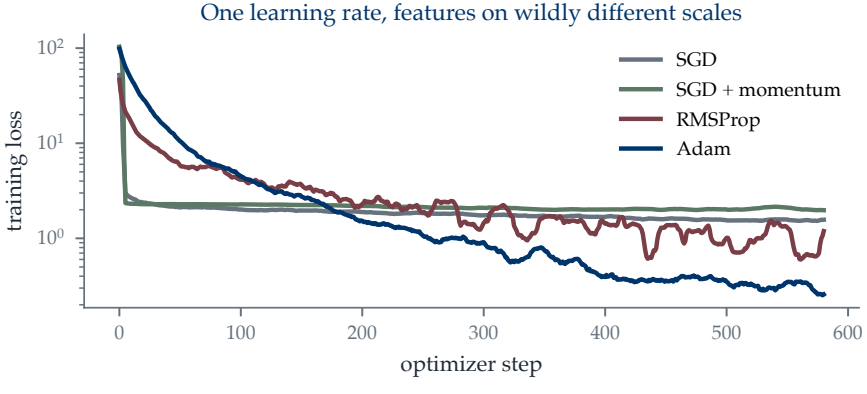
PyTorch `Adam(weight_decay)` with coupled L2 versus `AdamW`.

The Optimizers Store Different Histories

method	state per parameter	effective geometry
SGD	0	scalar learning rate
momentum	1	smoothed direction
AdaGrad	1	cumulative diagonal scale
RMSProp	1	recent diagonal scale
Adam / AdamW	2	direction + recent diagonal scale

For a large model, this state is a first-class memory cost; the end of the next section adds it up.

Where Adaptivity Actually Pays



A small network on digit pixels with one block of features stretched by 100, so the curvature is wildly unequal. At one shared learning rate, SGD reaches 1.57 and momentum 1.98 in 600 steps, RMSProp 1.19, and Adam 0.26: per-coordinate scaling is what makes a single η workable.

Identical initialization, batch 128, $\eta = 10^{-3}$ for every method.

Outline

1. Data Order, Minibatches, and Epochs
2. From Momentum to AdamW
3. Matrix-Aware Optimizers
4. The PyTorch Training Loop

Why Look at a Weight as a Matrix

Lecture 5 diagnosed slow training as unequal curvature: a step safe in the steep direction crawls in the flat one. AdaGrad and Adam answered by rescaling each coordinate by its own gradient history.

Why Look at a Weight as a Matrix

Lecture 5 diagnosed slow training as unequal curvature: a step safe in the steep direction crawls in the flat one. AdaGrad and Adam answered by rescaling each coordinate by its own gradient history.

But a layer's weight is a matrix $W \in \mathbb{R}^{m \times n}$, and so is its gradient G_t. A matrix has singular directions that no single entry names. If G_t has a few large singular values, the update moves almost entirely along those few directions and barely at all along the rest.

Why Look at a Weight as a Matrix

Lecture 5 diagnosed slow training as unequal curvature: a step safe in the steep direction crawls in the flat one. AdaGrad and Adam answered by rescaling each coordinate by its own gradient history.

But a layer's weight is a matrix $W \in \mathbb{R}^{m \times n}$, and so is its gradient G_t. A matrix has singular directions that no single entry names. If G_t has a few large singular values, the update moves almost entirely along those few directions and barely at all along the rest.

That is the Lecture 5 problem one level up: the unequal scale now sits across directions of the matrix, where a coordinatewise rule cannot see it.

Adam Treats a Weight Matrix Coordinatewise

For $W \in \mathbb{R}^{m \times n}$, Adam stores two $m \times n$ state matrices and rescales each entry on its own:

$$\Delta W_{jk}^{\text{Adam}} \propto -\frac{\widehat{M}_{jk}}{\sqrt{\widehat{V}_{jk}} + \varepsilon}.$$

The rows and columns of W are the output and input coordinates of a layer, and a diagonal preconditioner treats every entry as unrelated to its neighbors.

Muon, next, uses the matrix structure in the simplest way: it keeps the update's directions and discards its scale.

Muon Keeps the Directions and Drops the Scale

Start from a momentum matrix and its singular value decomposition,

$$M_t = \beta M_{t-1} + G_t, \qquad M_t = U \Sigma V^\top.$$

Muon replaces every singular value by one and steps along the result:

$$\boxed{W_{t+1} = W_t - \eta\, U V^\top}$$

Muon Keeps the Directions and Drops the Scale

Start from a momentum matrix and its singular value decomposition,

$$M_t = \beta M_{t-1} + G_t, \qquad M_t = U\Sigma V^\top.$$

Muon replaces every singular value by one and steps along the result:

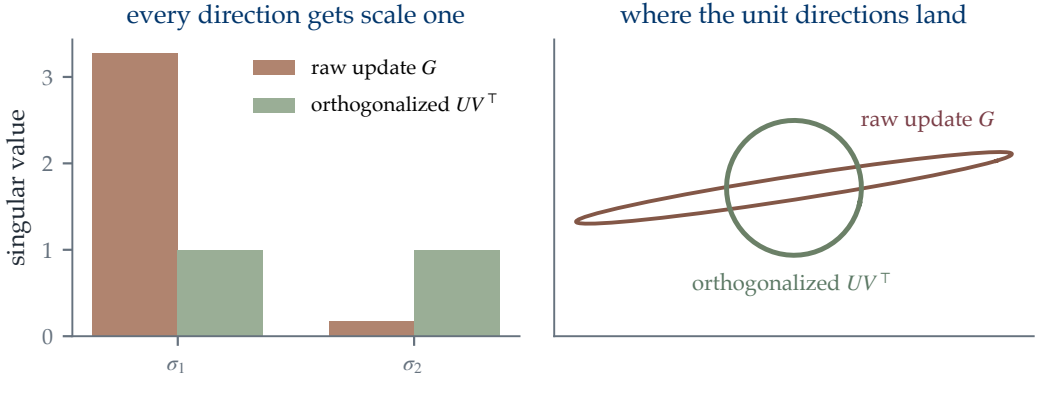
$$\boxed{W_{t+1} = W_t - \eta\, UV^\top}$$

Every singular direction of the update now moves by the same amount. For a 1×1 weight, $UV^\top = \operatorname{sign}(m_t)$, so Muon is sign descent lifted to matrices.

Code never forms the SVD: a few **Newton–Schulz** iterations of matrix products approximate $UV^\top$ (Appendix C). Muon serves the two-dimensional hidden weights; embeddings, biases, gains, and the output head use AdamW.

Source: Keller Jordan's Muon write-up (2024); Liu et al., 2025.

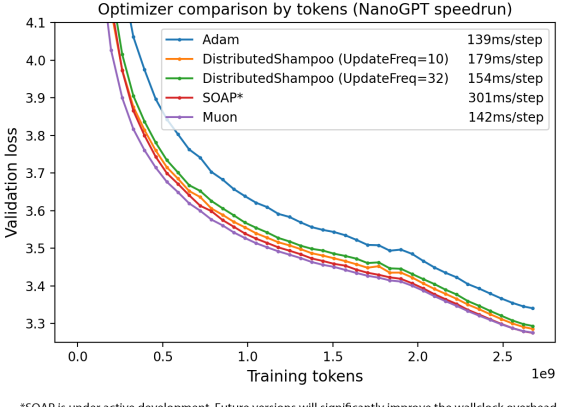
Orthogonalization Equalizes the Step



Left: the update's singular values, 3.27 and 0.17, both replaced by one. Right: where the same unit directions land after one step. The raw update stretches one direction nearly twenty times more than the other; the orthogonalized update moves equally in all of them.

Course calculation on a declared 2×2 update matrix.

Muon Has Promising but Conditional Evidence



One NanoGPT benchmark: Muon beats Adam at every token count, at nearly the same time per step (142 versus 139 ms).

Keller Jordan, “Muon,” 2024; attributed classroom excerpt.

Other Matrix-Based Optimizers

Shampoo = AdaGrad with the gradient history factored into a row matrix L_t and a column matrix R_t:

$$W_{t+1} = W_t - \eta\, L_t^{-1/4}\, G_t\, R_t^{-1/4}, \qquad L_t = \sum_{s \le t} G_s G_s^\top, \quad R_t = \sum_{s \le t} G_s^\top G_s.$$

SOAP = Adam with the same factorization: rotate the gradient into the eigenvectors Q_L, Q_R of L_t, R_t, take an Adam step, rotate back:

$$W_{t+1} = W_t - \eta\, Q_L\ \mathrm{Adam}(Q_L^\top G_t Q_R)\, Q_R^\top.$$

Appendix A explains both rules.

Optimizer Choice Remains an Empirical Procedure

Matrix-aware methods can improve time-to-quality in some transformer pretraining configurations, but the result depends on

- Parameter grouping and update scaling;
- Learning-rate and weight-decay tuning;
- Matrix shapes, batch size, and precision;
- Communication and kernel efficiency.

Use AdamW as a tuned baseline and compare under a declared compute and memory budget.

What a Training Run Holds in Memory

stored object	size, for P parameters	lifetime
parameters θ	P	whole run
gradients `p.grad`	P	one step
optimizer state	0 for SGD, P for momentum, $2P$ for AdamW	whole run
saved activations	batch size × total layer width	forward to backward

In float32 each number is 4 bytes, so AdamW holds 16 bytes per parameter before any activation: 3.8 MB for the MNIST MLP's 235,146 parameters, and 112 GB for a 7-billion-parameter model, more than one 80 GB accelerator. Mixed precision still keeps full-precision master weights and moments.

Activations are the one term the batch size controls and the only one freed within the step; `torch.no_grad()` skips them at evaluation.

Outline

1. Data Order, Minibatches, and Epochs
2. From Momentum to AdamW
3. Matrix-Aware Optimizers
4. The PyTorch Training Loop

Follow Along in the PyTorch Tutorials

The official **Learn the Basics** series builds the same pipeline on FashionMNIST, one page per piece:

tutorial page	where it appears in this lecture
Datasets & DataLoaders	`Dataset`, `DataLoader`, and the epoch (first section)
Autograd	`loss.backward()` filling `p.grad`
Optimizing Model Parameters	`train_loop` and `test_loop`: the next two slides
Save & Load Model	`state_dict()` for the model and the optimizer

`docs.pytorch.org/tutorials/beginner/basics/intro.html`
`docs.pytorch.org/tutorials/beginner/basics/optimization_tutorial.html`

Course notebooks, CPU or Colab: `first-neural-network` (a full training run) and `fashion-mnist-optimizers` (swapping optimizers).

The Tutorial's Setup

```
training_data = datasets.FashionMNIST(root="data", train=True,
                                      download=True, transform=transform)
train_dataloader = DataLoader(training_data, batch_size=64, shuffle=True)

model = NeuralNetwork()                       # 784 -> 512 -> 512 -> 10
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=1e-3)

for t in range(epochs):
    train_loop(train_dataloader, model, loss_fn, optimizer)
    test_loop(test_dataloader, model, loss_fn)
```

`shuffle=True` is our one addition; the tutorial's loader visits the batches in the same order every epoch. In the demo, five epochs of this SGD reach 65% test accuracy. Changing only the optimizer line to `torch.optim.AdamW` reaches 87%, at about five seconds per epoch on a laptop CPU.

The Tutorial's Training Loop

```
def train_loop(dataloader, model, loss_fn, optimizer):
    model.train()
    for batch, (X, y) in enumerate(dataloader):   # minibatch B_t
        pred = model(X)                           # forward pass
        loss = loss_fn(pred, y)                   # L_{B_t}(theta_t)
        loss.backward()                           # g_t -> every p.grad
        optimizer.step()                          # theta_{t+1}, state s_{t+1}
        optimizer.zero_grad()                     # empty p.grad for the next batch
```

One pass through `dataloader` is one epoch; one trip through the loop body is one optimizer step t. Zeroing after the step is the same as zeroing before the next backward call: each backward call starts from an empty gradient buffer.

Each Line Produces One Object

line	mathematical object	where it lives
`for X, y in dataloader`	minibatch $\mathcal{B}_t$	two tensors
`model(X)`	logits, plus saved activations	the autograd graph
`loss_fn(…)`	$L_{\mathcal{B}_t}(\theta_t)$	one scalar
`loss.backward()`	$g_t = \nabla_\theta L_{\mathcal{B}_t}(\theta_t)$	`p.grad` of every parameter
`optimizer.step()`	$(\theta_{t+1}, s_{t+1}) = \mathcal{U}(\theta_t, g_t, s_t; \eta_t)$	`p` and `optimizer.state`
`scheduler.step()`, if used	η_{t+1}	`optimizer.param_groups`

The pieces talk through narrow interfaces. The optimizer never sees the loss or the data; it reads `p.grad` and writes `p`. The loader never sees the model. This is why the data stream, the loss, and the update rule can each be swapped on its own.

Backward Adds into the Gradient Buffer

`loss.backward()` does not overwrite `p.grad`; it adds to it:

$$\texttt{p.grad} \longleftarrow \texttt{p.grad} + \nabla_p L.$$

Backward Adds into the Gradient Buffer

`loss.backward()` does not overwrite `p.grad`; it adds to it:

$$\boxed{\texttt{p.grad} \longleftarrow \texttt{p.grad} + \nabla_p L.}$$

Skip `zero_grad()` and after k loop bodies the buffer holds $g_1 + g_2 + \cdots + g_k$: the step uses a stale sum that grows with k, the loss still prints, and nothing crashes. That is why the clear comes first in the loop.

Backward Adds into the Gradient Buffer

`loss.backward()` does not overwrite `p.grad`; it adds to it:

$$\texttt{p.grad} \longleftarrow \texttt{p.grad} + \nabla_p L.$$

Skip `zero_grad()` and after k loop bodies the buffer holds $g_1 + g_2 + \cdots + g_k$: the step uses a stale sum that grows with k, the loss still prints, and nothing crashes. That is why the clear comes first in the loop.

The same behavior is a feature when used on purpose: several backward calls before one `step()` form the gradient of a larger batch without ever holding that batch in memory (gradient accumulation, Appendix D).

The Optimizer Object Holds Groups and State

```
matrices = [p for p in model.parameters() if p.ndim == 2]
vectors  = [p for p in model.parameters() if p.ndim < 2]    # biases, gains
optimizer = torch.optim.AdamW([
    {"params": matrices, "weight_decay": 0.1},
    {"params": vectors,  "weight_decay": 0.0},
], lr=1e-3, betas=(0.9, 0.95))

optimizer.param_groups[0]["lr"]   # one dict per group; schedulers write here
optimizer.state[p]                # buffers for p: step, exp_avg, exp_avg_sq
optimizer.zero_grad()             # set every p.grad to None
optimizer.step()                  # read p.grad; update p and its state
torch.save(optimizer.state_dict(), "opt.pt")   # groups and state together
```

Every `torch.optim` class has this shape: `param_groups` for hyperparameters, `state` for each parameter's history. A resumed run without `opt.pt` restarts Adam at $m = v = 0$, as if training began anew.

A Base Class Separates the Loop from the Rule

```
class MyOptimizer(torch.optim.Optimizer):
    @torch.no_grad()                          # the update is not part of the graph
    def step(self):
        for group in self.param_groups:       # one dict of hyperparameters per group
            for p in group["params"]:
                if p.grad is not None:        # zero_grad() set unused grads to None
                    self.update(p, p.grad, self.state[p], group)

    def update(self, p, g, state, group):     # the rule U(theta, g, s; eta)
        raise NotImplementedError
```

The constructor `torch.optim.Optimizer.__init__(params, defaults)` builds `self.param_groups`, filling the `defaults` into every group that does not set its own value, and `self.state`, an empty dictionary for each parameter. A subclass supplies the defaults and one `update` method: that method is the update rule $(\theta_{t+1}, s_{t+1}) = \mathcal{U}(\theta_t, g_t, s_t; \eta_t)$ of the second section, applied to one tensor.

AdamW Is One `update` Method

```
class MyAdamW(MyOptimizer):
    def __init__(self, params, lr=1e-3, betas=(0.9, 0.999), eps=1e-8, wd=0.01):
        super().__init__(params, dict(lr=lr, betas=betas, eps=eps, wd=wd))
    def update(self, p, g, st, group):
        lr, (b1, b2), eps, wd = (group[k] for k in ("lr", "betas", "eps", "wd"))
        if not st:                                          # first call: s_0
            st.update(t=0, m=torch.zeros_like(p), v=torch.zeros_like(p))
        st["t"] += 1
        t, m, v = st["t"], st["m"], st["v"]
        m.mul_(b1).add_(g, alpha=1 - b1)                    # m_t
        v.mul_(b2).addcmul_(g, g, value=1 - b2)             # v_t
        m_hat, v_hat = m / (1 - b1**t), v / (1 - b2**t)     # bias correction
        p.mul_(1 - lr * wd)                                 # decoupled decay
        p.addcdiv_(m_hat, v_hat.sqrt() + eps, value=-lr)
```

Each line is one AdamW formula. On the MNIST MLP this class matches `torch.optim.AdamW` to float32 rounding.

The Schedule Is a Function of the Step Count

With warmup length T_w and total optimizer steps T,

$$\eta_t = \eta_{\max} \times \begin{cases} t/T_w, & t \le T_w, \\ \frac{1}{2}\left(1 + \cos\left(\pi \, \frac{t - T_w}{T - T_w}\right)\right), & t > T_w. \end{cases}$$

```
def multiplier(t):                    # t counts optimizer steps
    if t < warmup_steps:
        return (t + 1) / warmup_steps
    progress = (t - warmup_steps) / (total_steps - warmup_steps)
    return 0.5 * (1 + math.cos(math.pi * progress))

scheduler = torch.optim.lr_scheduler.LambdaLR(optimizer, multiplier)
```

Here $T = E\lceil n/B \rceil$ counts optimizer steps, so a report must give $\eta_{\max}$, T_w, T, and the rule together, and say that t counts optimizer steps.

Choose the Peak Rate by a Controlled Sweep

Hold the model, data order, batch size, schedule shape, and step budget fixed. Run a short logarithmic grid such as $\eta_{\max} \in \{10^{-4}, 3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}\}$ and watch

$$\widehat{R}_{\text{train}}(t), \qquad \widehat{R}_{\text{val}}(t), \qquad \|g_t\|_2, \qquad \frac{\|\theta_{t+1} - \theta_t\|_2}{\|\theta_t\|_2}.$$

- A rate that diverges or spikes is too large, whatever its early speed;
- Among the stable rates, keep the one with the best validation loss at the end of the budget;
- A relative update of order 10^{-3} per step is a common healthy range; far smaller means the rate is wasted, far larger warns of instability.

Evaluation Runs the Forward Pass Without a Graph

```
def test_loop(dataloader, model, loss_fn):
    model.eval()
    test_loss, correct = 0, 0
    with torch.no_grad():
        for X, y in dataloader:
            pred = model(X)
            test_loss += loss_fn(pred, y).item()
            correct += (pred.argmax(1) == y).type(torch.float).sum().item()
    return test_loss / len(dataloader), correct / len(dataloader.dataset)
```

`model.eval()` switches dropout and batch statistics to their deterministic form (Lecture 11). `torch.no_grad()` tells autograd to save no activations, so evaluation uses a fraction of the training memory and can never touch `p.grad`. Validation changes no parameter, no optimizer state, and no layer statistic.

Summary: Data Stream and Steps

1. A `Dataset` maps an index to one example; a `DataLoader` turns one shuffled permutation into minibatches, and one pass is an epoch.
2. Steps, batches, and epochs are different units.
3. A learning-rate schedule must name which of those units indexes it.

$$g_t = \frac{1}{|\mathcal{B}_t|} \sum_{i \in \mathcal{B}_t} \nabla \ell_i(\theta_t), \qquad \text{steps per epoch} = \left\lceil \frac{n}{B} \right\rceil.$$

Summary: Optimizer State and Geometry

1. AdaGrad, RMSProp, Adam, and AdamW differ through the gradient history they store and how it rescales updates.
2. Shampoo and SOAP rescale the rows and columns of a weight matrix; Muon equalizes its singular directions. Each is a `torch.optim.Optimizer` whose `step()` reads `p.grad` and updates `p` and its state.
3. Parameters, gradients, optimizer state, and activations set the memory of a run, and memory decides which of these methods fits.

$$\Delta\theta_t^{\text{AdamW}} = -\eta_t \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon} - \eta_t \lambda \theta_t.$$

Next Lecture

The data stream and update rule now specify how parameters move. For images, the remaining choice is architectural: should every pixel location have an unrelated weight, or should local evidence be reused across positions?

Lecture 13 — Convolutional Neural Networks encodes locality and weight sharing, derives the resulting tensor shapes, and builds a complete image classifier with pooling, residual blocks, and batch normalization.

Reading for this lecture: D2L Chapter 12, Optimization Algorithms; DL Chapter 8; PyTorch *Learn the Basics*, especially Datasets & DataLoaders and Optimizing Model Parameters.

Appendix

Four self-contained parts for reading after class.

Part A. Shampoo and SOAP. Row and column preconditioners, why the powers are $-1/4$, and SOAP as Adam in Shampoo's eigenbasis.

Part B. Newer optimizers. Adafactor and Lion.

Part C. Muon in code. Newton–Schulz orthogonalization, a `MyMuon` class, and Muon and AdamW sharing one training loop.

Part D. Gradient accumulation. A large batch built from several small ones when the large batch does not fit in memory.

The `fashion-mnist-optimizers` notebook runs the Part C code.

Part A

Shampoo and SOAP

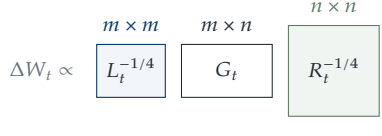
A.1 Shampoo Rescales Rows and Columns

Keep two running sums of the gradient matrix, one for each side:

$$L_t = \varepsilon I_m + \sum_{s \le t} G_s G_s^\top \;\; (m \times m), \qquad R_t = \varepsilon I_n + \sum_{s \le t} G_s^\top G_s \;\; (n \times n).$$

L_t records how the rows of past gradients co-vary, R_t the columns. The update multiplies the gradient on both sides:

$$\boxed{W_{t+1} = W_t - \eta\, L_t^{-1/4}\, G_t\, R_t^{-1/4}}$$



Source: Gupta, Koren, and Singer, 2018.

A.2 Why Quarter Powers: Check a One-by-One Weight

Let $m = n = 1$, so $G_s = g_s$ is a scalar. Then

$$L_t = R_t = \sum_{s \le t} g_s^2 \quad \Longrightarrow \quad L_t^{-1/4} g_t R_t^{-1/4} = \frac{g_t}{\sqrt{\sum_{s \le t} g_s^2}},$$

which is exactly AdaGrad. Each side carries the full gradient magnitude, so each takes a quarter root and the two together make the square root that AdaGrad divides by.

A.2 Why Quarter Powers: Check a One-by-One Weight

Let $m = n = 1$, so $G_s = g_s$ is a scalar. Then

$$L_t = R_t = \sum_{s \le t} g_s^2 \quad \Longrightarrow \quad L_t^{-1/4} g_t R_t^{-1/4} = \frac{g_t}{\sqrt{\sum_{s \le t} g_s^2}},$$

which is exactly AdaGrad. Each side carries the full gradient magnitude, so each takes a quarter root and the two together make the square root that AdaGrad divides by.

Compare with the ideal: AdaGrad on all mn entries jointly would store a $(mn) \times (mn)$ matrix. For $m = n = 1024$ that is 10^{12} numbers; L_t and R_t together hold 2×10^6.

The price is two matrix inverse fourth roots, taken in high precision and, in practice, refreshed only every few hundred steps.

A.3 In Its Own Eigenbasis, Shampoo Is a Diagonal Rescaling

Both accumulators are symmetric, so write $L_t = Q_L \Lambda_L Q_L^\top$ and $R_t = Q_R \Lambda_R Q_R^\top$ with orthogonal Q_L, Q_R. Then

$$L_t^{-1/4} G_t R_t^{-1/4} = Q_L \Big[\Lambda_L^{-1/4} \underbrace{(Q_L^\top G_t Q_R)}_{\widetilde{G}_t} \Lambda_R^{-1/4} \Big] Q_R^\top.$$

A.3 In Its Own Eigenbasis, Shampoo Is a Diagonal Rescaling

Both accumulators are symmetric, so write $L_t = Q_L \Lambda_L Q_L^\top$ and $R_t = Q_R \Lambda_R Q_R^\top$ with orthogonal Q_L, Q_R. Then

$$L_t^{-1/4} G_t R_t^{-1/4} = Q_L \Big[\Lambda_L^{-1/4} \underbrace{(Q_L^\top G_t Q_R)}_{\widetilde{G}_t} \Lambda_R^{-1/4} \Big] Q_R^\top.$$

In the rotated coordinates $\widetilde{G}_t$, entry (j,k) is simply divided by $(\lambda_j \lambda'_k)^{1/4}$. So Shampoo is three moves: rotate, divide each entry by a scale, rotate back.

A.3 In Its Own Eigenbasis, Shampoo Is a Diagonal Rescaling

Both accumulators are symmetric, so write $L_t = Q_L \Lambda_L Q_L^\top$ and $R_t = Q_R \Lambda_R Q_R^\top$ with orthogonal Q_L, Q_R. Then

$$L_t^{-1/4} G_t R_t^{-1/4} = Q_L \Big[\Lambda_L^{-1/4} \underbrace{(Q_L^\top G_t Q_R)}_{\widetilde{G}_t} \Lambda_R^{-1/4} \Big] Q_R^\top.$$

In the rotated coordinates $\widetilde{G}_t$, entry (j,k) is simply divided by $(\lambda_j \lambda'_k)^{1/4}$. So Shampoo is three moves: rotate, divide each entry by a scale, rotate back.

The rotation is what Adam lacks. The per-entry division is what Adam already does, with a running average and momentum on top. SOAP combines the two.

A.4 SOAP Runs Adam in Shampoo's Basis

1. Every k steps, update L_t, R_t and recompute their eigenvectors Q_L, Q_R.
2. Rotate the gradient: $\widetilde{G}_t = Q_L^\top G_t Q_R$.
3. Take an Adam step on $\widetilde{G}_t$:

$$\widetilde{M}_t = \beta_1 \widetilde{M}_{t-1} + (1-\beta_1)\widetilde{G}_t, \quad \widetilde{V}_t = \beta_2 \widetilde{V}_{t-1} + (1-\beta_2)\widetilde{G}_t^{\odot 2}, \quad \widetilde{\Delta}_t = \frac{\widehat{\widetilde{M}}_t}{\sqrt{\widehat{\widetilde{V}}_t} + \varepsilon}.$$

4. Rotate back and decay: $W_{t+1} = (1-\eta\lambda)W_t - \eta\, Q_L \widetilde{\Delta}_t Q_R^\top$.

A.4 SOAP Runs Adam in Shampoo's Basis

1. Every k steps, update L_t, R_t and recompute their eigenvectors Q_L, Q_R.
2. Rotate the gradient: $\widetilde{G}_t = Q_L^\top G_t Q_R$.
3. Take an Adam step on $\widetilde{G}_t$:

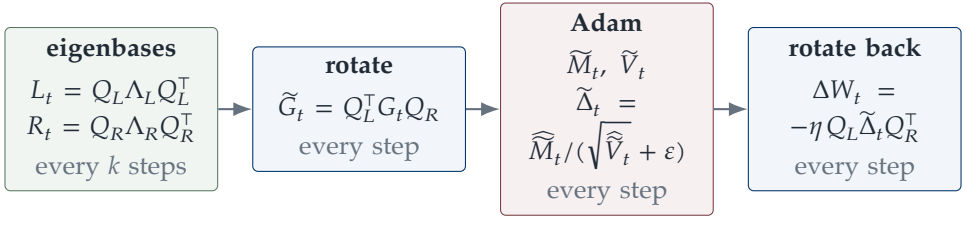
$$\widetilde{M}_t = \beta_1 \widetilde{M}_{t-1} + (1-\beta_1)\widetilde{G}_t, \quad \widetilde{V}_t = \beta_2 \widetilde{V}_{t-1} + (1-\beta_2)\widetilde{G}_t^{\odot 2}, \quad \widetilde{\Delta}_t = \frac{\widehat{\widetilde{M}}_t}{\sqrt{\widehat{\widetilde{V}}_t} + \varepsilon}.$$

4. Rotate back and decay: $W_{t+1} = (1-\eta\lambda)W_t - \eta\, Q_L \widetilde{\Delta}_t Q_R^\top$.

Two checks. With $Q_L = I_m$ and $Q_R = I_n$ this is AdamW. With $\sqrt{\widetilde{V}_t}$ replaced by the fixed scale $(\lambda_j \lambda'_k)^{1/4}$ it is Shampoo.

Source: Vyas et al. (2024), "SOAP: Improving and Stabilizing Shampoo using Adam".

A.5 SOAP, Step by Step



The expensive eigendecompositions run every few hundred steps; what runs every step is a rotation, the Adam update everyone already knows how to tune, and a rotation back. The state is Adam's two $m \times n$ moments plus L_t, R_t, Q_L, and Q_R.

Reported to reach a target loss in fewer steps than AdamW on language-model pretraining, at a modest wall-clock overhead.

A.6 Four Ways to Turn a Gradient Matrix into a Step

method	update for $W \in \mathbb{R}^{m \times n}$	state	extra work per step
AdamW	$\widehat{M}_t / (\sqrt{\widehat{V}_t} + \varepsilon)$, entrywise	$2mn$	elementwise
Shampoo	$L_t^{-1/4} G_t R_t^{-1/4}$	$m^2 + n^2$	inverse fourth roots
SOAP	Adam in the eigenbasis of L_t, R_t	$2mn + 2(m^2 + n^2)$	eigenbases every k steps
Muon	$UV^\top$ of the momentum M_t	mn	Newton–Schulz products

All four use the same gradient G_t. They differ in what they remember and in how much of the matrix structure the step respects. Block size, update frequency, numerical precision, and distributed sharding decide whether the geometry is affordable.

Part B

Newer Optimizers

B.1 Newer Optimizers Change One Ingredient

method	change to a known rule	state for $W \in \mathbb{R}^{m \times n}$
Adafactor	Adam with $\widehat{V}_t \approx r_t c_t^\top / (\mathbf{1}^\top r_t)$: row, column sums	$m + n$
Lion	step $-\eta \operatorname{sign}(\beta_1 M_{t-1} + (1-\beta_1) G_t)$	mn

Each keeps most of a method we have seen and changes one ingredient: Adafactor shrinks the memory of the second moment, and Lion replaces the direction by a sign. Asking which ingredient changes, and at what cost, is a good first reading of any new optimizer paper.

Source: Shazeer and Stern (2018); Chen et al. (2023).

Part C

Muon in Code

C.1 Newton–Schulz Computes $UV^\top$ Without an SVD

Scale $X_0 = M/\|M\|_F$, so every singular value lies in $(0, 1]$, and iterate

$$X_{k+1} = aX_k + b\,X_kX_k^\top X_k + c\,(X_kX_k^\top)^2X_k.$$

If $X_k = U\Sigma_kV^\top$, then $X_{k+1} = U\,p(\Sigma_k)\,V^\top$ with $p(\sigma) = a\sigma + b\sigma^3 + c\sigma^5$: the singular vectors never change, and only the singular values move toward 1.

```
def newton_schulz(M, steps=5, a=3.4445, b=-4.7750, c=2.0315):
    X = M / (M.norm() + 1e-7)                # singular values in (0, 1]
    for _ in range(steps):
        A = X @ X.T
        X = a * X + (b * A + c * A @ A) @ X   # sigma -> p(sigma)
    return X
```

Five steps send every $\sigma \geq 0.01$ into $[0.68, 1.21]$: an approximation built from matrix products alone. Library code transposes a tall M first, so that $XX^\top$ is the smaller Gram matrix.

C.2 Muon Is Another `update` Method

```
class MyMuon(MyOptimizer):
    def __init__(self, params, lr=0.02, beta=0.95, wd=0.0):
        super().__init__(params, dict(lr=lr, beta=beta, wd=wd))

    def update(self, p, g, st, group):
        M = st.setdefault("M", torch.zeros_like(p))
        M.mul_(group["beta"]).add_(g)                    # M_t = beta M_{t-1} + G_t
        O = newton_schulz(M)                             # approximately U V^T
        scale = max(1, p.size(0) / p.size(1)) ** 0.5     # larger step for tall W
        p.mul_(1 - group["lr"] * group["wd"])
        p.add_(O, alpha=-group["lr"] * scale)
```

The state is one matrix per weight, the *mn* of the A.6 comparison table, and the loop in `step()` is unchanged.

C.3 Two Optimizers Share One Loop

```
head = model.linear_relu_stack[-1].weight         # the tutorial model's output layer
hidden = [p for p in model.parameters() if p.ndim == 2 and p is not head]
others = [p for p in model.parameters() if p.ndim < 2 or p is head]
optimizers = [MyMuon(hidden, lr=0.02), torch.optim.AdamW(others, lr=1e-3)]

for x, y in loader:
    for opt in optimizers:
        opt.zero_grad()
    loss_fn(model(x), y).backward()               # one backward fills every p.grad
    for opt in optimizers:
        opt.step()                                # each updates only its own tensors
```

Muon takes the hidden weight matrices; AdamW takes biases, the output layer, and in a language model the embeddings. Each optimizer has its own learning rate and needs its own scheduler, and each one's `state_dict()` goes into the checkpoint.

Part D

Gradient Accumulation

D.1 Accumulation Builds a Large Batch from Microbatches

Split a batch of KB examples into K microbatches of B. With each microbatch loss divided by K, the buffer after K backward calls holds

$$g_t = \frac{1}{K} \sum_{k=1}^{K} g_{t,k},$$

the gradient of the mean loss over all KB examples. Then call `optimizer.step()` once.

- Loader batches processed: K; backward calls: K;
- Optimizer steps: 1, so the schedule advances by one.

The memory that forced the split is the saved activations, which grow with the batch size; each microbatch frees its own after its backward call.

D.2 Accumulation in PyTorch

```
optimizer.zero_grad()
for k, (x, y) in enumerate(loader):
    loss = loss_fn(model(x), y) / accum_steps
    loss.backward()
    if (k + 1) % accum_steps == 0:
        optimizer.step()
        scheduler.step()
        optimizer.zero_grad()
```

Steps per epoch fall from $\lceil n/B \rceil$ to about $\lceil n/B \rceil / K$, so a schedule written in optimizer steps must use the smaller count. Batch-dependent layers such as batch normalization see B examples at a time and can behave differently from a true batch of KB.